

Fundamental Programming Techniques

Assignment 2 - Support Presentation (Part 2)

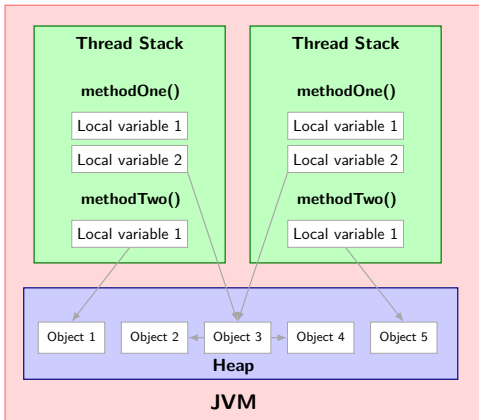
Computer Science Department, TUCN

Spring 2026

Agenda

- Java Memory Model
- Volatile Variables
- Synchronized Statements
- Synchronized Methods
- Atomicity
- Synchronized Collections
- Concurrent Collections

Java Memory Model (JMM)



Source: Jakob Jenkov, Java Memory Model,
<https://jenkov.com/tutorials/java-concurrency/java-memory-model.html>

- JVM is the runtime environment for Java programs
- Heap is the memory area where objects are stored
- JMM controls how threads access shared memory
- Threads use local working memory while shared data is in the heap
 - ▶ **Atomicity** - operations complete entirely or not at all
 - ▶ **Visibility** - changes made by one thread are seen by others
 - ▶ **Ordering** - the sequence in which operations appear to execute across threads

Volatile Variables

Important

The Java `volatile` keyword guarantees **visibility of changes** to variables across threads.

Read from Heap

For every working read request for a volatile variable, a thread reads the value from the main memory (Heap).

Write to Heap

For every write request for a volatile variable, a thread writes the value to the main memory(Heap).

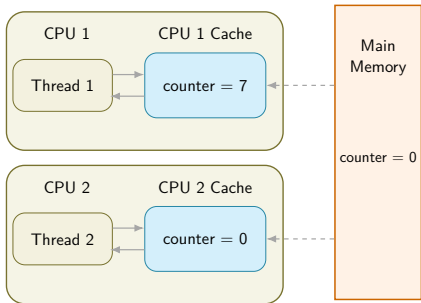
- A thread does not cache the value of a volatile variable in its working memory.
- Using a volatile variable is useful only in multi-threaded environment for variables that are shared among threads.
- It is faster and cheaper than using a synchronized block.

Volatile Variables

Not using volatile variables

```

1 public class SharedObject {
2     public int counter = 0;
3 }
    
```



Source

There exists a **VISIBILITY** Problem

Using volatile variables

```

1 public class SharedObject {
2     public volatile int counter = 0;
3 }
    
```

If thread **T1** modifies the counter, and thread **T2** reads the counter (but never modifies it), declaring the counter variable as **volatile** is enough to guarantee visibility for **T2** of writes to the counter variable.

If both **T1** and **T2** were incrementing the counter variable, then declaring the counter variable volatile would not have been enough.

Volatile Variables

- **Problem:** If two threads are **both reading and writing** to a shared variable, then using the `volatile` keyword for that is not enough!
- **Solutions:**
 - 1 Use the **synchronized keyword around critical sections** to guarantee that the reading and writing of the volatile variable is atomic
 - 2 Use **atomic data types** found in `java.util.concurrent` package (e.g., **AtomicLong**, **AtomicReference**)
- Performance considerations:
 - ▶ Reading and writing of volatile variables causes the variable to be read or written to the main memory
 - ▶ Reading from and writing to main memory is more expensive than accessing the CPU cache

Use volatile variables when you really need to enforce visibility of variables!

Synchronized Statements

- To preserve state consistency, update variables in a single atomic operation
- **Block of code to be guarded by the lock:**

```
1    synchronized (lock) { // Reference to an object  
2        // Access or modify shared state guarded by lock  
3        ...  
4    }
```

- Every Java object can implicitly act as a lock for purposes of synchronization
 - ▶ **intrinsic locks** or **monitor locks**
 - ▶ Automatically acquired and released
 - ▶ A happens-before relationship established

Synchronized Statements

- Examples of synchronized blocks

```

1    synchronized(MyClass.class) {
2        // some code
3    }
4    // Or
5    synchronized(this) {
6        // some code
7    }

```

Locks can be used to create a synchronized code

```

1    public void transfer(Account a, Account b, double sum) {
2
3        synchronized (a) { // Thread 1 locks account a
4
5            // Thread 2 locks account b
6
7            synchronized (b) {
8
9                // transfer sum from a to b
10           }
11       }
12
13   }
14
15   }
16   ...

```

Deadlock situation!!!

```

1    // Thread 1
2    transfer(a, b, sum1);
3
4    // Thread 2
5    transfer(b, a, sum1);

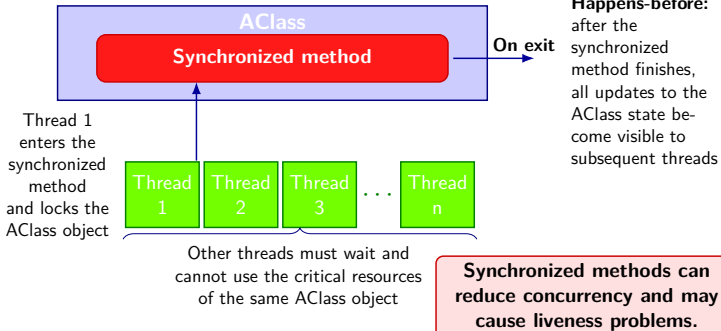
```

Locks can lead to deadlocks

Synchronized Methods

Key Points

- **synchronized** is written in the method declaration
- **Constructors cannot be synchronized**
- Method exit guarantees visibility of updates



Warning

Synchronized methods can cause **liveness problems**.

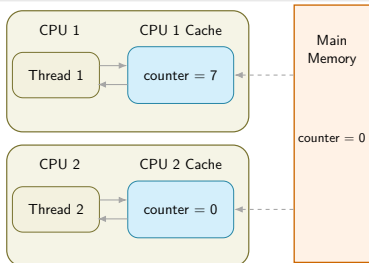
Atomicity

- Atomic action cannot be interleaved => avoids thread interference
- `java.util.concurrent.atomic` package

Reads and writes are atomic for reference variables and for most primitive variables (all types except long and double)

Reads and writes are atomic for all variables declared `volatile` (including long and double variables)

Source



Working with memory doesn't happen instantly

Atomicity

```
1      int i = 0;  
2      i++; // Get I value and increment it
```

Race Conditions for i++ compound operations

i being accessed simultaneously by both Thread1 and Thread2, it can lead to inconsistencies:

- result can be 1 if both threads got 0 and incremented to 1
- result can be 2, if second thread got the value 1 incremented by the first thread

i++ is not atomic

- read-modify-write operation
- not stateless and is not thread-safe due to instance variable

```
1      // Atomic Operations  
2      AtomicInteger i = new AtomicInteger(0);  
3      i.getAndIncrement();
```

Volatile variables

- Changes are always visible to other threads
- Establishes a happens-before relationship with subsequent reads of that same variable
- Sees also the side effects of the code that led up the change

Synchronized Collections

- Synchronization wrappers which create synchronized views of collections. Examples: `synchronizedCollection`, `synchronizedList`, `synchronizedMap`, etc

```
1 List<String> list = Collections.synchronizedList(new ArrayList());
```

- Achieve thread-safety through intrinsic locks
- Synchronized collections are thread safe

```
1 Collection c = Collections.synchronizedCollection(myCollection);
2 ...
3 synchronized(c) {
4     Iterator i = c.iterator(); // Must be in the synchronized block
5     while (i.hasNext())
6         foo(i.next());
7 }
```

Regarding previous usage of `Iterator`, it must manually synchronize on the returned collection when iterating over it

Concurrent Collections

- Designed for concurrent access from multiple threads => `java.util.concurrent` package contains: `BlockingQueue`, `ConcurrentHashMap`, `ConcurrentNavigableMap`, `CopyOnWriteArrayList`
- Achieve thread-safety
 - ▶ **BlockingQueue**
 - ★ provides blocking put and take methods
 - ★ support the producer-consumer design patterns
 - ▶ **ConcurrentHashMap**
 - ★ divides its data into segments
 - ★ different threads can acquire locks on each segment
 - ★ Multiple threads can access the map at the same time
 - ▶ **CopyOnWriteArrayList**
 - ★ creates a separate copy of List for each write operation

These concurrent collections are coming with a greater performance than synchronized collections